

项目深度面试题

七大技术模块

- 1. 商家主页 Redis 缓存（Cache Aside + 防三缓）
- 2. Spring Security + JWT 认证与鉴权
- 3. 在线预约与档期维护
- 4. 抢档期排队（Lua + Redis ZSET）
- 5. 微服务架构（Gateway + Nacos + Dubbo/Feign）
- 6. 实时消息推送与通知提醒（含社区 Feed 流 + RabbitMQ）
- 7. 按需搜索推荐（Elasticsearch + Functon Score）

一、商家主页 Redis 缓存（Cache Aside + 防三缓）

Cache Aside 是什么？读写流程是什么？

Cache Aside（旁路缓存）是最常用的缓存模式，应用层手动管理缓存，数据库是唯一权威数据源。

读流程：

- ① 先查缓存
 - ├ 命中 (hit) → 直接返回，不查 DB
 - └ 未命中 (miss) → 查 DB → 将结果写入缓存（设 TTL）→ 返回

写流程：

- ① 更新数据库
- ② 删除缓存（不是写入新值）
 - 下次有请求时，走读流程的 miss 分支，从 DB 拉最新值重建缓存

四种缓存写策略横向对比：

策略	写操作	读操作	优点	缺点
Cache Aside	写 DB → 删缓存	miss 时查 DB 再回填	只缓存真正被读的数据，实现简单	有短暂不一致窗口；冷启动时缓存为空，首次请求全 miss

策略	写操作	读操作	优点	缺点
Write Through	同步写缓存 + 写 DB（串行）	直接读缓存	写后立即一致，读永远命中	写延迟高（两次 IO 串行）；不热的数据也占缓存空间
Write Back (Write Behind)	只写缓存，异步批量刷盘	直接读缓存	写性能最高，适合高频写	断电/宕机丢数据；实现复杂，需保证刷盘顺序
Write Around	只写 DB，绕过缓存	miss 时查 DB 再回填	避免一次性写入的热点污染缓存	写后首次读必然 miss，延迟高

项目为什么选 Cache Aside + 删缓存：

- 1. **按需缓存**：商家信息读多写少，Cache Aside 只缓存被读到的数据，Write Through 会把所有商家（包括从不被访问的）都推进 Redis，浪费内存。
- 2. **写频率低**：商家更新信息是低频操作，不需要 Write Through 或 Write Back 的"高写性能"优化，额外的复杂度不值得。
- 3. **Write Back 有数据丢失风险**：商家评分等数据必须持久化，异步刷盘不可接受。
- 4. **删而不是更新缓存**：见 Q4，并发写时"更新缓存"会产生覆盖问题，删除是更安全的选择。

Q1：商家主页为什么要加缓存？直接查数据库有什么问题？

代码位置： MerchantServiceImpl.getDetail()

商家主页是高频读、低频写的场景：用户反复查看同一商家详情，但商家信息（简介、价格、评分）很少变动。每次直接查 MySQL 有两个代价：一是 DB IO 延迟（几十毫秒），在高并发下会造成明显的响应时延；二是数据库连接数有限，热门商家的流量会打满连接池，影响其他业务。

加 Redis 缓存后，命中率高的情况下从几十毫秒降到个位数毫秒，且对数据库完全透明。

↳ 追问：Redis 是单线程的，为什么还能支撑高并发？

Redis 的"单线程"指命令处理线程只有一个，但这正是它高性能的来源：单线程避免了锁竞争和上下文切换开销。网络 IO 层用 epoll/kqueue 实现 IO 多路复用，一个线程同时监听成千上万个连接，只在有数据可读时才处理。加之 Redis 所有操作都在内存完成，没有磁盘 IO，单线程每秒可处理 10 万以上命令。

Q2：getDetail() 里同时防了穿透、击穿、雪崩三个问题，分别是什么场景，各自怎么解决的？

这三个问题发生条件不同，解法也独立，分开理解。代码位置： MerchantServiceImpl.getDetail()

穿透：请求的 key 在 Redis 和 DB 都不存在（比如攻击者构造大量不存在的 merchantId），每次都打穿到 DB，DB 扛不住。

解法：不存在的 id 也写入缓存，值是空字符串，TTL 2 分钟（短，防止合法商家注册后长时间不可见）。下次命中这个空值哨兵直接抛 `MERCHANT_NOT_FOUND`，不再查 DB。

击穿：某个热门 key 缓存刚好过期，瞬间大量并发请求同时发现缓存缺失，全部穿透到 DB，造成 DB 瞬时压力峰值（雪崩的单 key 版本）。

解法：Redisson 分布式锁 `lock:merchant:{id}`，只有一个线程能查 DB 并重建缓存，其他线程在锁上排队。拿到锁后还要**再查一次缓存**（DCL），因为在等锁期间前一个线程可能已经写好缓存了。

雪崩：大量 key 同时过期（比如所有 key 用了相同 TTL），同时大量请求打到 DB。

解法：TTL 加随机偏移 `25 + random.nextInt(11)` = 25~35 分钟随机，让不同 key 的过期时间错开，避免集中过期。

↳ **追问：空值哨兵的 TTL 为什么设 2 分钟，而不是和正常缓存一样的 30 分钟？**

空值哨兵代表"当前这个 id 不存在"，但这个结论会过时：商家注册后 id 就从"不存在"变成"存在"了。如果 TTL 设 30 分钟，新注册的商家要等 30 分钟才能被搜到，业务上不可接受。2 分钟是"防穿透效果"与"新数据可见延迟"的平衡点——2 分钟内的穿透攻击被挡住，正常用户最多等 2 分钟看到新商家。

Q3：防缓存击穿时，光有分布式锁还不够，为什么还需要在锁内再查一次缓存？（即 `getDetail()` 里的 DCL 是怎么回事）

Q2 的答案里说"解法是分布式锁 + DCL"，这道题就是在追问 DCL 这半部分。代码位置：

```
MerchantServiceImpl.getDetail()
```

```
// 第一次查缓存（无锁，所有请求并发执行）
String cached = stringRedisTemplate.opsForValue().get(cacheKey);
if (cached != null) { return ...; } // 命中直接返回，不抢锁

// 缓存未命中，加分布式锁（同一 merchantId 的请求在此排队）
lock.lock();

// 第二次查缓存（加锁后、查 DB 前）
cached = stringRedisTemplate.opsForValue().get(cacheKey);
if (cached != null) { return ...; } // 前一个线程已重建好，直接用

// 真正查 DB（只有第一个拿到锁的线程会走到这里）
```

光有锁，没有第二次查缓存会怎样：

假设缓存刚过期，10 个请求同时发现缓存是 null，都通过了第一次检查，随后排队等锁。请求1 拿到锁、查 DB、写缓存、释放锁。请求2 拿到锁后——如果直接查 DB，那锁只是把并发变成了串行，10 个请求还是全打到 DB，毫无意义。

有了第二次查缓存：请求2 拿到锁后先再读一次 Redis，发现请求1 已经写好了，直接返回，不再查 DB。后续 8 个请求同理。**DB 只被查了一次。**

第一次检查和第二次检查的分工：

	第一次（锁外）	第二次（锁内）
目的	正常命中时快速返回，不排队抢锁	复用前一个线程重建好的缓存，避免重复查 DB
有无锁保护	无（并发执行，性能最好）	有（此时只有一个线程在跑）

↳ 追问：这个 DCL 和 Java 双重检查锁（DCL 单例）是同一个模式吗？原理上有什么共同点？

本质相同，都是"在获取锁之前和之后各做一次条件检查"。Java DCL 单例中，锁外检查避免每次创建都抢锁，锁内检查避免多个线程都通过了锁外检查后重复初始化。这里的逻辑完全一致：锁外快速返回已缓存数据，锁内防止排队的线程都查 DB。Java DCL 还需要 `volatile` 来禁止指令重排，因为对象初始化不是原子的；而这里的 Redis `set` 和 `get` 各自是原子操作，不需要类似的内存屏障。

Q4：商家信息更新后，为什么选"改 DB → 删缓存"，而不是"改 DB → 更新缓存"或"先删缓存 → 再改 DB"？

代码位置：`MerchantServiceImpl.updateInfo()`，最后一行 `stringRedisTemplate.delete(cacheKey)`

共有三种常见写策略，逐一分析：

方案一：改 DB → 更新缓存（直接写新值进缓存）

并发写时有覆盖问题：

线程A：更新 DB（新值=5）
线程B：更新 DB（新值=6）
线程B：更新缓存（写入6）
线程A：更新缓存（写入5）← 旧值覆盖了新值！

DB 存的是 6，缓存存的是 5，不一致。根本原因：两次写 DB 和两次写缓存的顺序无法保证一致。

方案二：先删缓存 → 再改 DB

有"读旧值回填"问题：

线程A：删除缓存
线程B：读缓存，miss，查 DB（此时 DB 还是旧值）
线程B：将旧值写入缓存
线程A：更新 DB（新值写入）

缓存里现在是旧值，DB 是新值，不一致。且这个旧值会一直待在缓存里直到 TTL 过期，窗口期比较长。

方案三（当前实现）：改 DB → 删缓存

线程A: 更新 DB (新值)
线程A: 删除缓存
线程B: 读缓存, miss → 查 DB, 拿到新值 → 回填缓存

删除比写入"安全": 删除操作幂等, 无论顺序如何, 最终状态都是缓存不存在, 下次请求从 DB 拿最新值。代价是缓存重建期间有一次穿透, 但结合分布式锁防击穿, 只有一个线程查 DB, 可以接受。

方案三也存在极小概率的不一致窗口 (见 Q5), 但窗口极短, 商家信息场景完全可接受。

↳ 追问: 方案三还是存在极小概率的不一致 (Q5 提到的窗口), 有没有能完全消除的方案?

理论上可以用"延迟双删"缓解: 改 DB 后立即删缓存, 再等几百毫秒后第二次删缓存, 把 Q5 里那个短窗口内回填的旧值也清掉。但"延迟"的时间难以准确估计, 且仍有极小窗口。要做到强一致, 只能引入分布式事务 (如 Seata), 保证 DB 写入和缓存删除要么都成功要么都回滚, 代价是引入 2PC 的性能损耗和实现复杂度。对于商家详情这种场景, 轻微延迟一致性完全可接受, 强一致代价远超收益。

Q5: Cache Aside 防三缓这套方案有没有缺点? 能做到强一致性吗?

接 Q2-Q4, 整体评价这套方案的边界。

不能做到强一致性, 存在一个短暂的不一致窗口:

线程A: 读缓存, miss
线程A: 查 DB, 拿到旧值 (新值还没提交)
线程B: 更新 DB, 提交新值
线程B: 删除缓存
线程A: 写缓存, 写入旧值 ← 此时缓存存了旧值, 但 DB 已经是新值

这个窗口极短 (几毫秒)。对于商家详情这种场景, 轻微的延迟一致性完全可接受——用户看到的评分稍微滞后几毫秒不影响体验。

如果要求强一致性, 需要引入分布式事务 (2PC/TCC) 或者直接不缓存, 代价远超收益。

↳ 追问: 如果业务上真的需要强一致, 比如金融场景, 应该怎么处理?

有三条路: ① 直接不缓存, 每次读 DB, 牺牲性能换一致性; ② 引入 Seata 分布式事务 (AT 模式), 框架层面保证 DB 和 Redis 操作在同一个全局事务里, 代价是每次写操作增加多次网络交互; ③ 用 DB 事务 + Outbox 模式, 把"删缓存"这个动作记录进 DB, 由独立 relay 服务保证"DB 写成功则缓存最终一定被删"。哪种方案取决于对性能和一致性的权重, 金融场景通常选① 或 Seata。

二、Spring Security + JWT 认证与鉴权

Q23：JWT 登录的完整流程是什么？token 存在哪里，后续请求怎么验证？

代码位置： UserServiceImpl.buildLoginVO() + JwtAuthenticationFilter + SecurityConfig

登录时：

```
// UserServiceImpl.buildLoginVO()
vo.setToken(JwtUtil.generate(user.getId())); // 生成 JWT，直接返回，不写 Redis
```

JwtUtil.generate() 生成三段式 JWT： Header.Payload.Signature 。Payload 里自包含 sub (userId) 和 exp (7 天后过期时间) ，用 HS256 对称签名。不写 Redis，服务端零状态存储。前端收到后存入 localStorage。

后续请求：

① 前端：Axios 拦截器把 JWT 放入 Header "token"

② JwtAuthenticationFilter.doFilterInternal():

```
token = request.getHeader("token")
userId = JwtUtil.parse(token) // 验签 + 解析，无 IO，纯计算
SecurityContextHolder 写入 UsernamePasswordAuthenticationToken(userId)
```

③ SecurityConfig 路由规则：

- 白名单路径 (permitAll) → 放行 (即使 context 空)
- 受保护路径 + context 空 → AuthenticationEntryPoint 返回 401

Service 层： SecurityUtil.getCurrentUserId() 从 SecurityContextHolder (内部是 ThreadLocal) 读 userId, O(1), 无 IO。

请求结束： Spring Security 的 SecurityContextHolderFilter 自动清除 ThreadLocal，防线程归还到池后数据残留。

与原 Sa-Token 方案的关键区别：

	Sa-Token (原方案)	Spring Security + JWT (现方案)
token 生成	UUID 随机串	自包含 JWT (Header.Payload.Signature)
服务端存储	Redis (token → userId)	无 (完全无状态)
验证方式	查 Redis 取 userId (有 IO)	验签解析 (纯计算，无 IO)
主动失效	支持 (删 Redis key)	不支持 (7天过期前无法撤销)
多服务共享	共享同一 Redis 实例	共享同一 Secret Key，各自验签

↳ 追问：JWT 不存 Redis，那 token 在 7 天过期前，如果用户被封号或者修改了密码，怎么让 token 立刻失效？

这是 JWT 无状态的固有代价。有两种常见方案：① Redis 黑名单：封号时把该 token 的 jti (JWT ID) 写入 Redis，每次验签后额外查一次 Redis 黑名单，命中则拒绝。缺点是引入了 IO，丧失了“纯计算验证”的优势，且黑名单随时间积累会变大（需按 exp 定期清理）；② 缩短 token 有效期 + refresh token：access token 有效期设 15 分钟，配合有效期 7 天的 refresh token 静默续签。封号时拒绝 refresh token 续签，最多 15 分钟后 access token 自动失效。项目当前未实现，是已知缺陷。

Q24：为什么 JWT token 放 Header 而不放 Cookie？两种方式的安全边界是什么？

接 Q23，追问 token 传输方式的设计取舍。

Cookie 默认跨域受限 (SameSite=Strict)，且 Cookie 是浏览器自动携带的，有 CSRF 风险——攻击者诱导用户访问恶意页面，浏览器自动带上 Cookie，伪造合法请求。

Header token 需要 JavaScript 主动读取和设置，CSRF 攻击无法控制 JS 行为（同源策略），天然防 CSRF。移动端也不依赖 Cookie，逻辑更统一。

代价是 JS 必须能访问 localStorage (HttpOnly Cookie 无法被 JS 读)，XSS 攻击可以窃取 token，所以必须严格防 XSS。两种方案都有安全风险，只是防的方向不同。

↳ 追问：HttpOnly Cookie 是怎么防 XSS 的？如果用了 HttpOnly Cookie，CSRF 怎么防？

HttpOnly 标志让 Cookie 对 JavaScript 不可见，document.cookie 读不到它，XSS 脚本无法窃取。这是 Cookie 方案相对 localStorage 的安全优势。但 HttpOnly Cookie 仍然会被浏览器自动携带，所以 CSRF 问题依然存在，需要配合 CSRF Token (SameSite=Strict 或在请求里加一个 double-submit cookie) 来防御。两种方案本质上是防 XSS 和防 CSRF 之间的取舍，没有同时完美防两者的方案，只能根据业务场景选优先级。

Q25：项目密码原来用 MD5 存储，有什么安全缺陷？已经怎么修复的？

代码位置： UserServiceImpl.register() / UserServiceImpl.login()

MD5 的问题：

MD5 是确定性函数，相同明文永远生成相同哈希，且项目里没有加盐：

```
// 原来（有安全缺陷）
user.setPassword(SecureUtil.md5(dto.getPassword())); // 注册
user.getPassword().equals(SecureUtil.md5(dto.getPassword())); // 登录
```

攻击者可以预先计算海量常见密码的 MD5（彩虹表），数据库一旦泄露，直接查表反推明文。"123456" 的 MD5 是固定值，彩虹表秒破。

修复方案：改用 BCrypt

BCrypt 的核心优势：

1. **内置随机盐**：每次 `encode()` 自动生成不同的盐，相同密码产生完全不同的哈希，彩虹表完全失效
2. **盐存在哈希结果里**：哈希字符串本身包含了盐（`$2a$10$<22位盐><31位哈希>`），`matches()` 时自动提取盐来比较，不需要额外存盐字段
3. **计算成本可调**：`new BCryptPasswordEncoder(12)` 中的 12 是 work factor，值越大算得越慢，可以随硬件提升而调高，对抗暴力破解

改动只有两处代码，极低成本：

```
// 注册：encode() 内部自动生成随机盐，每次结果不同
user.setPassword(passwordEncoder.encode(dto.getPassword()));

// 登录：matches() 从哈希字符串里提取盐值后再比较，不能用 equals
if (!passwordEncoder.matches(dto.getPassword(), user.getPassword())) {
    throw new BusinessException("密码错误");
}
```

为什么登录不能用 `equals` 而必须用 `matches`：

BCrypt 哈希格式是 `$2a$10$<盐><哈希>`，每次注册生成的盐不同，所以同一个密码每次 `encode()` 出来的字符串都不一样。`matches()` 先从存储的哈希里解析出盐，用同一个盐对用户输入重新计算，再比较哈希部分。直接 `equals` 永远不相等。

↳ 追问：BCrypt 既然这么好，为什么业界还有人用 Argon2？BCrypt 有什么不足？

BCrypt 设计于 1999 年，是 CPU 密集型算法，但现代 GPU 可以大规模并行破解 CPU 密集型哈希。Argon2（2015 年密码哈希竞赛冠军）是内存密集型算法，每次哈希需要消耗大量内存（可配置），GPU 并行破解的优势被内存带宽限制抵消，抗 GPU/ASIC 暴力破解能力更强。Spring Security 5.x 开始支持 Argon2。项目用 BCrypt 对当前规模完全够用，但如果是高安全要求的场景（如金融、医疗），Argon2 是更现代的选择。

三、在线预约与档期维护

预约创建完整流程

代码位置：`BookingController + BookingServiceImpl.create() + ScheduleServiceImpl`

① 客人查看商家主页月历（`listByMonth`），选择可用档期

② 客人填写问卷（拉取 `QuestionnaireTemplate`），点击“发起预约”

- ③ 后端：Redisson 分布式锁防并发重复提交（见 Q43）
- ④ 锁内：Double-Check 查 DB，确认该用户未预约该档期
- ⑤ 校验档期状态（是否开放、是否已满）
- ⑥ 创建 Booking (status=0, 待确认)，答案存入 questionnaire_answers 字段
- ⑦ 商家在“收到的预约”列表看到新记录 → confirm() / cancel()
- ⑧ confirm(): status 0→2, 发 MQ 通知客人
- ⑨ 超时未确认：XXL-Job 扫描自动 cancel（见 Q45）

Q43：客人发起预约时，如何防止同一用户并发重复提交？

代码位置：BookingServiceImpl.create()，Redisson 分布式锁部分

客人快速双击或网络重传可能导致同一请求在极短时间内到达两次。如果不加保护，会创建出两条重复预约。

解决方案：Redisson 分布式锁 + 锁内双重校验（DCL）

```
String lockKey = "booking:create:" + userId + ":" + scheduleId;
RLock lock = redissonClient.getLock(lockKey);
boolean locked = lock.tryLock(3, 30, TimeUnit.SECONDS);
// 3s 等不到锁直接失败，持锁最多 30s 防业务异常导致死锁

if (!locked) throw new BusinessException("操作频繁，请稍后再试");

try {
    // 锁内：再查一次 DB，确认该用户未预约该档期
    Booking exist = bookingMapper.selectOne(
        Wrappers.<Booking>lambdaQuery()
            .eq(Booking::getUserId, userId)
            .eq(Booking::getScheduleId, scheduleId)
    );
    if (exist != null) throw new BusinessException(ErrorCode.BOOKING_DUPLICATE);

    // 通过校验，创建预约
    ...
} finally {
    if (lock.isHeldByCurrentThread()) lock.unlock();
    // isHeldByCurrentThread() 防止持锁超时后再 unlock 抛异常
}
```

lock key 为什么同时包含 userId 和 scheduleId：

- 不同档期的预约完全独立，不需要互相等锁
- 只有同一用户预约同一档期才需要串行化
- 锁粒度最小化，最大化并发吞吐

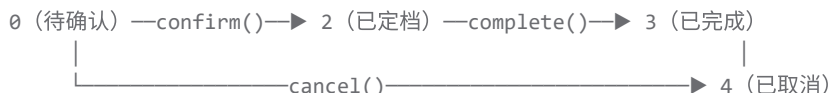
↳ 追问：leaseTime 设 30 秒，如果业务只需要 200ms，这 30 秒会有什么隐患？Redisson 的看门狗机制是什么？

leaseTime 是“锁持有上限”，防止 JVM 崩溃时锁永远不释放（死锁）。正常情况 200ms 执行完后 finally 块手动 unlock，30 秒的 leaseTime 不会有影响。如果不显式设 leaseTime（传 -1），Redisson 会启动“看门狗（Watchdog）”——默认每

隔 10 秒自动续期一次（续到 30 秒），只要持锁线程还活着就一直续。JVM 崩溃后续期停止，Redis 的 key 等 30 秒自动过期释放锁。两种方式都能防死锁，显式 leaseTime 更简单，看门狗更适合执行时间不确定的长事务。

Q44：预约状态机是怎么设计的？为什么 status=1（待付款）不使用？

代码位置：BookingController 的 confirm/complete/cancel 接口，BookingServiceImpl 对应实现



状态 1（待付款）在数据库枚举中保留，但当前未使用——项目没有接入支付，confirm() 直接将 status 从 0 设为 2，跳过付款环节。保留 1 是为了后续集成支付时可以插入 0→1→2 的流转，不需要改数据库 schema。

每次状态变更的合法性校验：每个操作前先查当前 status，如果不在允许的前置状态内，抛 BOOKING_STATUS_ERROR (ResultCode=20003)：

```
// complete() 只允许从"已定档"流转
if (booking.getStatus() != 2) throw new BusinessException(ResultCode.BOOKING_STATUS_ERROR);
```

↳ 追问：状态机校验在 Service 层做，如果两个线程并发调用 confirm() 会有问题吗？如何解决？

会有竞态：两个线程都查到 status=0，都通过了校验，都执行 UPDATE，最终执行了两次状态变更（虽然结果一样都是 2）。更危险的是并发 cancel 和 confirm——一个设为 4，一个设为 2，最终状态取决于哪个 UPDATE 后执行，与业务预期不符。解法是让 UPDATE 语句携带前置状态条件：UPDATE booking SET status=2 WHERE id=? AND status=0，然后检查 affected rows。如果 affected rows=0，说明被并发改了，抛异常。这是 CAS（Compare-And-Set）思想在数据库层的应用，比加分布式锁更轻量。

Q45：超时未确认的预约怎么自动取消？为什么用 XXL-Job 而不是本地 @Scheduled？

代码位置：BookingJobHandler.java，XXL-Job 注册名 cancelTimeoutBookingJob

XXL-Job 执行流程：

1. 定时（如每 10 分钟）扫描 status=0（待确认）且 create_time < NOW() - INTERVAL 48 HOUR 的预约
2. 批量 UPDATE status=4（已取消）
3. 对每条预约调用 MQSender.sendBookingCancelled()，NotifyConsumer 消费后通过 WebSocket 通知客人

为什么不用 `@Scheduled`：

在微服务多实例部署下，每个 `mhp-booking` 实例都运行 `@Scheduled`，3 个实例同时扫描会导致同一批预约被重复取消（重复写 `status`、重复发通知）。

XXL-Job 是分布式调度平台：同一任务在同一时刻只分配给一个执行器实例执行，其余实例静默等待。执行器实例的地址通过 Nacos 注册中心发现，新实例上线自动加入，故障实例自动摘除。

↳ 追问：批量 UPDATE 几百条记录，会不会造成 MySQL 长时间锁等待影响其他业务？怎么优化？

UPDATE WHERE 条件走索引（id 或复合索引）时，锁的是行锁不是表锁，但如果一次 UPDATE 几百条，这几百个行锁会同时持有到事务提交，期间其他事务读这些行需要等待。优化方案是分批执行：每次 UPDATE LIMIT 100，提交事务，释放行锁，再执行下一批。这样把一个大事务拆成多个小事务，单次持锁时间从“几百行”缩短到“100 行”，大幅降低对其他业务的影响。代价是整个扫描任务耗时增加，但定时任务对时延不敏感，完全可接受。

Q46：档期月历是怎么查询的？

代码位置：`ScheduleController.listByMonth()`，`ScheduleServiceImpl.listByMonth()`

```
GET /api/schedule/month?merchantId=42&year=2026&month=9
```

SQL 核心：

```
SELECT * FROM schedule
WHERE merchant_id = ?
AND DATE_FORMAT(date, '%Y-%m') = ?
```

返回该商家当月所有档期列表，前端按 `date` 字段渲染日历格子：

- 无档期的日期：灰色，不可点
- 有档期且可预约：绿色，点击进入预约对话框
- 已满/已关闭：标记颜色，不可点

↳ 追问：DATE_FORMAT 函数在 WHERE 条件里，会不会导致索引失效？怎么优化？

会导致索引失效。`DATE_FORMAT(date, '%Y-%m')` 是函数调用，MySQL 无法用 `date` 列上的索引，需要全表扫描。优化方案：把函数移到右侧，用范围查询代替：`WHERE merchant_id = ? AND date >= '2026-09-01' AND date < '2026-10-01'`。这样 `(merchant_id, date)` 联合索引可以完整使用，先定位商家再按日期范围筛选，复杂度从全扫描到 $O(\log n)$ 。

Q47：问卷模板是怎么设计的？客人的填写答案存在哪里？

代码位置： QuestionnaireTemplate 实体， BookingCreateDTO

商家在后台创建问卷模板（ QuestionnaireTemplate ）：定义题目和题型（文本/单选/多选），绑定到该商家账号。

客人预约时：

1. 前端调 GET /api/questionnaire/merchant/{merchantId} 拉取模板字段
2. 渲染动态表单，客人填写
3. 答案随 BookingCreateDTO 的 answers 字段（JSON 格式）一起提交
4. 后端将 answers JSON 字符串存入 booking.questionnaire_answers 列

选 JSON 存答案而不是单独建答案表：问卷答案是附属于预约的只读快照，不需要按字段独立查询，JSON 存储足够简单高效。

↳ 追问：答案存 JSON 列，如果将来要按答案内容搜索（比如“找所有填了‘北京’的预约”），怎么查？

MySQL 5.7+ 可以用 JSON 函数： WHERE JSON_EXTRACT(questionnaire_answers, '\$.city') = '北京' ，但这是全表扫描，性能差。如果这种查询是高频需求，有两条路：① 把关键答案字段（如城市）提取成独立列，加普通索引；② 把数据同步到 Elasticsearch，用 term 查询。项目当前问卷答案只用于展示给商家，不做搜索，JSON 存储足够。如果将来有搜索需求，按需提取列是最低成本的方案。

四、抢档期排队（Lua + Redis ZSET）

抢档期为什么用 Redis 而不用 MySQL？有其他业界方案吗？

抢档期的核心难题是**瞬时并发写**：开放时间一到，大量用户在同一秒发请求，每个请求都要原子完成三件事——检查是否已入队、检查队列是否已满、写入并返回名次。

MySQL 做不好的原因：

1. FOR UPDATE 锁竞争：所有并发请求争抢同一批行锁，大量事务排队等待
2. 连接池耗尽：每个事务持有一个 DB 连接，连接池通常只有几十到两百个，千人同时抢直接打满
3. 磁盘 IO：MySQL 即使有 buffer pool，索引写入最终落盘，比 Redis 纯内存慢一到两个数量级
4. 没有原生排名操作：MySQL 查“当前排第几”需要额外一次 COUNT(*) 扫描，Redis ZSET 的 ZRANK 是 $O(\log n)$

四种业界方案对比：

方案	并发能力	一致性	实现复杂度	适用场景
MySQL FOR UPDATE	低（连接池瓶颈）	强	低	低并发
Redis Lua + 直写 MySQL	高	最终一致	中	当前方案，中等并发
MQ 排队	极高	最终一致	高	大流量秒杀（12306、双十一）

方案	并发能力	一致性	实现复杂度	适用场景
Redis 预检 + MQ 落库	极高	最终一致	高	生产级秒杀，当前方案升级版

MQ 排队方案（淘宝秒杀标准思路）：

用户请求 → 写入 RocketMQ/Kafka → 单消费者串行处理 → 写 MySQL → WebSocket 推结果给用户

MQ 天然 FIFO，消费者串行处理彻底消除并发冲突，不需要任何锁。缺点是引入异步，用户需要等待结果。

Redis 预检 + MQ 落库（当前方案的升级版）：

用户请求 → Redis Lua 原子预检（去重、判满、记录名次）
├ 失败（已入队/队满）→ 直接返回
└ 成功 → 发 MQ → 消费者写 MySQL（ACK 保证可靠）

Redis 做高速过滤和排名，MQ 解决 Redis 成功但 MySQL 失败的不一致问题（见 Q15）。这是待优化方向。

Q12：抢档期为什么要用 Lua 脚本？不能用 Redisson 锁或者 Redis 事务（MULTI/EXEC）吗？

代码位置：`rush.lua + ScheduleServiceImpl.rush()`，Lua 脚本由 `stringRedisTemplate.execute(rushScript, ...)` 执行。

最直接的原因：命令之间有时间窗口，并发必然出 bug

假设队列还剩 1 个名额，两个用户同时发请求：

用户A ZSCORE → nil（未入队）	用户B ZSCORE → nil（未入队）
用户A ZCARD → 9（未滿）	用户B ZCARD → 9（未滿）
用户A ZADD 成功	用户B ZADD 成功 ← 实际超额！

ZSCORE 和 ZADD 之间有时间窗口，两人都通过了判满检查，实际超出 `maxQueueSize`。Lua 脚本在 Redis 单线程上原子执行，三步整体不可分割，从根本上消除这个竞态。

分布式锁方案：可行，但代价高

每个用户需要 5 次网络往返，且 Java 线程在等锁期间阻塞：

Java 线程1：SET NX 抢锁 → ZSCORE → ZCARD → ZADD → DEL 释放锁（5次网络）
Java 线程2：SET NX 抢锁失败 → Java 线程阻塞等待，占用线程池...

高并发下线程池被大量阻塞线程占满，新请求无法处理。

Lua 脚本方案每个用户只需 **1 次网络往返**：发脚本 → 等结果。Redis 内部串行执行脚本，每次几微秒，对 Java 侧透明。Java 线程不阻塞，只是在等网络 RTT。

Redis 事务（MULTI/EXEC）方案：无法做条件判断

MULTI/EXEC 的工作方式是"先排队，后批量执行"，EXEC 之前所有命令只是入队，没有被执行，因此 ZSCORE 的结果在 MULTI 和 EXEC 之间完全拿不到。无法在 Java 侧写"如果已入队就不 ZADD"这样的条件逻辑。

要实现条件写必须用 WATCH（乐观锁），但高并发下大量线程会遇到 WATCH 失败，每次失败都要重新 WATCH → 重读 → 重试，形成**重试风暴**——越多人抢，失败越多，重试越多，性能反而越差。

Lua 方案：三步原子，脚本内直接读中间值做判断

```
local existScore = redis.call('ZSCORE', KEYS[1], ARGV[1])
if existScore then return -1 end -- 已入队
if redis.call('ZCARD', KEYS[1]) >= tonumber(ARGV[3]) then return 0 end -- 队满
redis.call('ZADD', KEYS[1], tonumber(ARGV[2]), ARGV[1])
return redis.call('ZRANK', KEYS[1], ARGV[1]) + 1 -- 返回名次
```

三种方案对比：

方案	网络往返/用户	Java 线程阻塞	高并发表现
分布式锁	5 次	等锁期间阻塞	线程池耗尽，吞吐低
MULTI/EXEC + WATCH	多次（含重试）	重试时反复请求	重试风暴，越忙越慢
Lua 脚本	1 次	不阻塞	吞吐高，延迟稳定

↳ 追问：Lua 脚本在 Redis 里是"原子"的，这个原子和数据库事务的 ACID 原子性是同一个概念吗？

不完全相同。Redis Lua 的"原子"仅指执行过程不可被其他命令插入（单线程串行保证），没有回滚语义——如果 Lua 脚本执行到一半出错（比如类型错误），已执行的命令不会撤销。数据库事务的原子性包含回滚：事务内任何步骤失败，所有已执行操作都会被撤回，保证"要么全做要么全不做"。此外，数据库事务的原子性是 ACID 的一部分，还配套有持久化（D）和隔离（I）；Redis Lua 没有这些保证。简单说：Redis Lua 原子 = 不可打断；DB 事务原子 = 不可打断 + 可回滚 + 持久化。

Q13：排队数据为什么用 Redis ZSET 而不用 LIST？

接 Q12，追问 rush.lua 里选 ZSET 的原因。

ZSET（有序集合）的每个成员有一个 score，这里用时间戳毫秒作为 score，实现"先来先得"：

操作	LIST	ZSET
去重（一人只能入队一次）	需要 LPOS，O(n)	ZSCORE 判断，O(log n)
查询排名	需要遍历，O(n)	ZRANK，O(log n)
先来先得顺序	插入顺序，但无法防止重复	score=时间戳，天然有序

ZRANK 返回的排名就是用户的队列位置，直接展示"您排在第 N 位"，无需额外计算。

↳ 追问：ZSET 内部是什么数据结构？为什么 ZRANK 是 O(log n) 而不是 O(1)？

ZSET 内部有两种编码：元素少（默认 128 个以内）且 member 较短时用 listpack（紧凑型双向链表，节省内存）；元素多时用跳表（skiplist）+ 哈希表。哈希表用于 O(1) 的 ZSCORE（按 member 查 score），跳表用于 O(log n) 的有序操作（ZRANK/ZADD/ZRANGE）。ZRANK 无法做到 O(1) 是因为需要在有序结构里找到 member 的位置，跳表的多层索引让这个查找是 O(log n)，类似二分查找，无法更快。

Q14：rush.lua 返回值为什么用 -1 / 0 / 正整数这种编码，不直接返回字符串？

接 Q12，追问脚本的返回值设计。

返回值同时承载"成功/失败"和"成功时的数据"两个语义：

- -1 ：已入队（业务正常情况，不是系统错误）
- 0 ：队列已满（业务正常情况）
- >0 ：排队名次，从 1 开始（ZRANK 返回 0-based，脚本内 +1 转成用户友好的 1-based）

Java 侧只需判断返回值落在哪个区间，不需要额外字段。选整数而不是字符串，是因为 Lua 返回整数最简单，Redis execute 的 Long 类型也更好处理，不需要再解析字符串。

↳ 追问：Lua 函数能返回的类型有哪些限制？如果想返回一个包含名次和时间戳的复合对象，怎么做？

Redis Lua 脚本的返回值只能是以下类型：integer（整数）、bulk string（字节串）、array（数组，元素也是这些类型）、nil、error。不支持直接返回 JSON 对象或浮点数（浮点会被截断为整数）。要返回复合对象，可以用 array：return {rank, timestamp}，Java 侧收到 List<Object>，按下标取值。也可以序列化成 JSON 字符串（bulk string）返回，Java 侧再反序列化，但增加了序列化开销。本项目只需要返回一个整数，编码最简单。

Q15：Lua 脚本写 Redis 成功后，才去写 MySQL 的 RushRecord，两边数据会不一致吗？

代码位置：ScheduleServiceImpl.rush()，Lua execute 之后紧跟 rushRecordMapper.insert(record)。

会有，且是真实的一致性缺口，不只是"短暂窗口"。

Lua 成功（Redis ZSET 已有该用户）后，Java 才调 `rushRecordMapper.insert(record)` 写 MySQL。如果这一步抛异常，会出现：

- Redis ZSET：用户在队列（ZSCORE 有值）
- MySQL `rush_record`：没有这条记录

后果：用户再次调用 `rush()` 时，Lua 的 ZSCORE 返回有值 → 返回 `-1`（"您已在队列中"），但查排队记录什么都查不到，行为矛盾。

两种修法：

简单补偿（成本低）：catch DB 异常后执行 `ZREM key userId`，撤回入队：

```
try {
    rushRecordMapper.insert(record);
} catch (Exception e) {
    stringRedisTemplate.opsForZSet().remove("schedule:" + scheduleId, String.valueOf(userId));
    throw new BusinessException("抢档期失败，请重试");
}
```

极小概率 ZREM 也失败（网络抖动），Redis 和 MySQL 仍不一致，但概率极低，可接受。

Outbox 模式（生产级）：在同一个本地事务里同时写 `rush_record` 和 `outbox`（消息表），独立 relay 服务扫描 `outbox` 发 MQ、标记已处理。核心思路：把"DB 写入"和"后续操作"放进同一个本地事务，用数据库的 ACID 代替分布式事务，不引入分布式事务的复杂度。

现状：项目目前没有补偿机制，这是已知的一致性缺口，应在后续迭代修复。

↳ **追问：用 ZREM 撤回时，ZREM 本身也可能失败，这个问题能彻底解决吗？**

用 ZREM 撤回是"尽力而为"，ZREM 失败则留下孤立的 ZSET 记录，无法在这个方案内彻底解决。彻底解决需要换思路：以 MySQL 为唯一权威，定期对账——扫描 Redis ZSET 里的所有 member，对照 MySQL `rush_record`，找出"在 ZSET 但不在 MySQL"的记录，执行 ZREM 清理。这是"定期补偿"而非"实时补偿"，允许短暂不一致，最终对齐。或者彻底弃用 Redis ZSET 存排队状态，改为只在 MySQL 里存，用 `SELECT COUNT FOR UPDATE` 串行化，牺牲并发性能换强一致性。

Q39：抢档期开放前，怎么做缓存预热？

背景：`rushOpenTime` 存在 `schedule` 表里，时间已知，属于可预期的热点事件。

1. **XXL-Job 定时任务：**每隔几分钟扫描 `rushOpenTime` 在未来 10 分钟内的档期，提前将对应商家信息写入 Redis（`merchant:info:{id}`）。开放时间一到，所有请求直接命中缓存。
2. **ZSET 无需提前初始化：**抢档期的 Redis ZSET（`schedule:{scheduleId}`）在第一次 ZADD 时 Redis 自动创建，不需要提前建 key。可以提前校验 `maxQueueSize` 配置是否合理，避免开抢时才发现配置错误。

3. 服务启动时预热： `ApplicationRunner` 在启动时加载当天及近期有开放计划的档期，防止服务刚重启就遇到抢档期流量打穿缓存。

↳ 追问：预热的数据写入 Redis 后，如果商家在开抢前修改了档期信息，缓存里还是旧数据，怎么处理？

这是预热的固有问题：预热时间越早，开抢前被修改的概率越大。解法有两条：① 缩短预热窗口（比如从"提前 10 分钟"改为"提前 3 分钟"），减少修改窗口期；② 商家修改档期信息时，主动删除对应的 Redis 缓存 key（Cache Aside 写策略），下次有请求时重建缓存。两条可以结合：正常写流程走 Cache Aside 删缓存，预热只做首次填充，这样预热和正常写流程不冲突，预热的数据如果被修改会自动失效。

Q40：热点数据怎么识别？抢档期场景里具体怎么做？

通用方案（运行时探测）：监控每个 Redis key 的访问频率，超过阈值则标记热点（Redis 4.0 的 `redis-cli --hotkeys`，或应用层自定义计数器），再对热点做保护（本地缓存或更短 TTL）。

项目不需要运行时探测——热点是可预期的：`rushOpenTime` 精确到分钟，提前知道哪个档期、哪个时间点是热点，直接利用这个信息主动预热（见 Q39），不等运行时才识别。

运行时探测适合热点不可预期的场景（微博热搜、突发爆款商品）。你的场景属于"已知热点"，主动预热比被动探测更简单、更可靠。

↳ 追问：本地缓存（Caffeine）和 Redis 配合使用时，如果本地缓存的数据过期，但 Redis 里还有，会有几次网络往返？如果本地缓存数据和 Redis 数据不一致怎么处理？

本地缓存 miss → 查 Redis（1次网络往返）→ 命中则直接写入本地缓存返回，共 1 次网络往返。如果 Redis 也 miss，再查 DB 并回填 Redis 和本地缓存。不一致的场景：Redis 数据更新（DB 写入后删 Redis key），但本地缓存还持有旧值，直到本地缓存 TTL 过期前都会返回旧数据。处理方式：① 本地缓存设短 TTL（如 5 秒），接受短暂不一致；② Redis 写入时广播失效事件（通过 MQ 或 Redis Pub/Sub），各实例收到事件后主动清除本地缓存；③ 部署实例数少时直接不用本地缓存，Redis 未成瓶颈不值得引入二级缓存的复杂度。

五、微服务架构（Gateway + Nacos + Dubbo/Feign）

Q26：Gateway 和 Nginx 在项目里各自负责什么？为什么不让 Nginx 直接转发到微服务？

请求链路在 CLAUDE.md 里的"微服务架构 → 请求链路"一节有完整描述。

	Nginx	Spring Cloud Gateway
处理对象	前端静态文件（HTML/JS/CSS）+ SSL	/api/** 业务请求

	Nginx	Spring Cloud Gateway
路由依据	写死 IP:Port	Nacos 服务名 (lb://mhp-account)，自动服务发现
新增实例	需改 upstream 配置 + reload	Nacos 自动感知，无需改动
扩展能力	SSL 终止、静态文件缓存、高吞吐	鉴权过滤器、限流、路由规则热更新

```
用户 → Nginx (443/80)
├─ 静态文件 → 直接返回 dist/
└─ /api/** → proxy_pass → Gateway:80
    → 按路径匹配分发到 mhp-account/booking/social
```

Nginx 直接转发到微服务的问题：新增实例或服务迁移都要手动改 Nginx 配置；也无法做服务发现和负载均衡。Gateway 通过 Nacos 动态感知实例列表，扩容时无需改任何配置。

↳ 追问：Spring Cloud Gateway 底层用的是 Servlet 模型还是 WebFlux？为什么 API 网关适合用响应式模型？

Spring Cloud Gateway 基于 WebFlux (Reactor + Netty)，是非阻塞 IO 模型。API 网关的特点是：大量并发连接、每个请求主要时间花在等待下游服务响应 (IO 等待)、本身几乎不做 CPU 密集计算。传统 Servlet 模型一连接一线程，高并发下线程数暴涨，线程切换和内存开销巨大；WebFlux 用少量线程 (通常等于 CPU 核数) 通过事件循环处理所有连接，线程不在 IO 等待期间阻塞，资源利用率极高，正好匹配 API 网关的访问模式。

Q27：项目里 Dubbo RPC 和 OpenFeign 并存，选哪个的依据是什么？

```
代码位置： BookingServiceImpl (同时注入了 AccountFeignClient 和 @DubboReference
RpcMerchantService )
```

具体划分：

- **Dubbo**：用于权限校验和状态校验这类"失败直接终止"的强依赖—— getMerchantByUserId (判断是否为商家)、getBookingById (校验预约状态)、updateMerchantScore (评分更新)
- **Feign**：用于批量数据拉取 (batchGetUsers 、 batchGetMerchants) 这类可以降级返回空集合的弱依赖

这个划分在 mhp-booking 和 mhp-social 里都适用——两个服务都同时注入了 Feign Client 和 @DubboReference ，按语义分别使用。

↳ 追问：Dubbo 底层用什么协议？和 HTTP 相比性能上有什么区别？

Dubbo 默认使用 Dubbo 协议 (基于 TCP, Hessian2 二进制序列化)。对比 HTTP：① 协议头更紧凑，HTTP 头部是文本格式，每次携带大量 Header，Dubbo 协议头固定 16 字节；② 序列化更高效，Hessian2 是二进制序列化，比 JSON (文本) 体积小、解析快；③ 长连接复用，Dubbo 在服务间维持长连接池，避免每次请求的 TCP 三次握手开销。代价是 Dubbo 协议不通用——前端浏览器不能直接调，只能用于服务间通信，所以必须经过 Gateway 层转换。Feign 走 HTTP，跨语言、跨系统兼容性更好，这是它适合外部暴露的原因。

Q28：为什么服务间调用方向保持单向无环（DAG）？如果出现循环调用会怎样？

当前调用方向：

```
mhp-booking → mhp-account  
mhp-social → mhp-account  
mhp-social → mhp-booking
```

如果 mhp-account 也调 mhp-booking，形成环： booking → account → booking → ... ，可能导致：

1. 调用链无限递归：A 调 B，B 调 A，直到线程栈溢出或超时。
2. 启动顺序死锁：A 依赖 B，B 也依赖 A，谁先启动都找不到对方。
3. 级联雪崩：A 慢导致 B 等待，B 又在等 A，全部超时。

单向无环（DAG）保证：任意服务出问题，只影响"上游"依赖它的服务，不会反向影响"下游"服务，故障半径可控。

↳ 追问：如果业务上真的出现了双向依赖（A 需要 B 的数据，B 也需要 A 的数据），怎么破除循环而不丢失功能？

有三种思路：① 事件驱动解耦——A 不直接调 B，而是发 MQ 事件，B 订阅处理后发另一个事件，A 再订阅。双方都只依赖 MQ，没有直接服务依赖；② 提取公共服务——把 A 和 B 都需要的数据放进第三个服务 C，A 和 B 都调 C，C 不调任何人；③ 重新划分服务边界——循环依赖往往说明这两个服务的职责没有划清，考虑把循环依赖的部分合并到同一个服务内处理，用本地方法调用代替跨服务调用。实践中最常用的是①（事件驱动）和③（合并边界）。

Q29：评价提交时需要校验预约状态，mhp-social 怎么拿到 mhp-booking 的数据？

接 Q27，具体说明评价这条链路的跨服务调用。代码位置： ReviewServiceImpl.create()

通过 Dubbo RPC。RpcBookingService 接口定义在 mhp-common，Provider 在 mhp-booking，Consumer（mhp-social）注入：

```
@DubboReference(version = "1.0.0")  
private RpcBookingService rpcBookingService;
```

调用 rpcBookingService.getBookingById(orderId) 拿到 BookingDTO，走 Dubbo 协议点对点直连，不经过 Gateway，也不需要配置路由白名单。

选 Dubbo 而不是 Feign 的原因：校验预约状态是评价创建的前置条件，失败则整个业务终止，没有降级空间，适合强依赖的 RPC 调用语义。

↳ 追问：Feign 和 Dubbo 调用失败（超时/宕机）时，默认行为分别是什么？Feign 的 fallback 怎么配？

Feign 默认超时抛 `FeignException`，可以通过 `@FeignClient(fallback = XxxFallback.class)` 配置降级类，在降级类里返回默认值（如空列表）。Dubbo 默认超时 1 秒抛 `RpcException`，可以配置 `mock = "force:return null"` 做 mock 降级，或者 `@DubboReference(mock = "com.xxx.XxxMock")` 指定降级实现。本项目评价校验是强依赖，没有配置 fallback，失败直接抛异常回滚事务，让用户重试。批量拉用户信息（Feign）则配了降级：返回空 map，列表页展示时用“已注销”占位。

Q16：评价创建的完整流程是什么？为什么要加 `@Transactional`？

代码位置：`ReviewServiceImpl.create()`

- ① 从 `SecurityContext` 取当前用户 ID
- ② Dubbo RPC 调 `mhp-booking`，验证预约存在且属于当前用户
- ③ 检查预约 `status=3`（已完成），只有完成的预约可评价
- ④ 检查是否已评价（一个预约只能评一次）
- ⑤ 写入 `review` 表
- ⑥ 重新聚合该商家所有评价，计算新的平均分
- ⑦ Dubbo RPC 调 `mhp-account`，更新商家 `avgScore` 和 `reviewCount`
- ⑧ `mhp-account` 删除商家缓存，触发下次访问重建

加 `@Transactional` 覆盖步骤 ⑤~⑦：review 写入和商家评分更新必须同时成功或同时回滚。如果 review 写入成功但评分更新失败（比如 `mhp-account` 宕机），下次查商家详情评分就对不上实际评价，数据不一致。

局限：`@Transactional` 管不了 Dubbo RPC 的远程事务——`mhp-account` 的 DB 更新不在这个事务里。真正的强一致需要分布式事务（Seata），这里用的是最终一致：review 写入成功后，account 更新即使短时失败，可以触发补偿重试，评分最终会对齐。

↳ 追问：`@Transactional` 能管住 Dubbo RPC 的远程调用吗？如果 `mhp-account` 宕机，review 写入会回滚吗？

不能。`@Transactional` 只管本服务的数据库事务，Dubbo RPC 调用的是远程服务，完全独立的数据库连接和事务，不在 Spring 事务管理范围内。如果 `mhp-account` 宕机，Dubbo 调用抛 `RpcException`，Spring 捕获异常后回滚本服务的 review 写入，用户看到报错。这是“保守策略”——宁可让用户重试，也不让 review 和评分长期不一致。真正的跨服务强一致需要引入 Seata（TCC 模式），代价是每次都要协调多个服务的预留/提交/回滚，性能损耗和实现复杂度都很高，当前业务规模不值得。

Q17：updateMerchantScore() 为什么每次全量重新算平均分，不做增量计算？

接 Q16，追问 `ReviewServiceImpl.updateMerchantScore()` 里 `reviews.stream().mapToInt(...).average()` 的实现选择。

增量计算的方式：
$$\text{newAvg} = (\text{oldAvg} * \text{oldCount} + \text{newScore}) / (\text{oldCount} + 1)$$
。

看起来简单，但有一个隐患：如果有评价被删除或修改，增量公式就失效了——需要不同的增量公式。随着业务变复杂（删评价、修改评价、投诉撤销评价），增量逻辑越来越难维护。

全量计算直接从 DB 聚合，永远正确，无论中间发生什么操作。代价是每次多一次 `SELECT ... WHERE merchant_id=?`，但商家评价数量通常在千级以内，查询耗时毫秒级，完全可接受。这是"以简单换正确"的取舍。

↳ 追问：如果某个商家积累了 10 万条评价，每次评价提交都全量 `COUNT/AVG` 会不会成为性能瓶颈？

会。 `SELECT AVG(score), COUNT(*) FROM review WHERE merchant_id = ?` 在 10 万行时需要全扫这 10 万行，即使走了 `merchant_id` 索引，聚合计算也要几十毫秒。优化方案：在 `merchant` 表冗余 `avg_score` 和 `review_count` 两列（项目已经这样做了），每次新增评价后增量更新这两列即可，不需要全量聚合。删评价和改评价时触发一次全量重算（低频操作，可接受），正常新增评价（高频操作）走增量计算。这是"读写分离"思想：把聚合结果缓存在主表列，牺牲少量写一致性换取读性能。

Q18：评价写入后要更新 mhp-account 的评分，如果 mhp-account 宕机会怎样？有没有更好的方案？

接 Q16，追问跨服务强依赖的容错边界。

当前实现：Dubbo RPC 调用失败会抛异常，`@Transactional` 回滚 `review` 写入，用户看到报错。

这是**保守策略**——宁可让用户重试，也不让 `review` 和评分长时间不一致。对当前业务规模可以接受。

更健壮的方案是 **MQ 异步解耦**：`review` 写入成功后，发一条"新评价"消息到 RabbitMQ，`mhp-account` 消费后更新评分。即使 `account` 短暂宕机，消息持久化在 MQ 里，恢复后继续消费，最终一致。代价是引入 MQ，架构复杂度上升，这是项目的待优化点。

↳ 追问：改成 MQ 异步后，用户提交评价看到"成功"，但评分可能几秒后才更新，这段时间其他用户看到的还是旧评分，这种最终一致性能接受吗？有没有让用户感知不到延迟的办法？

业务上可以接受：评分更新延迟几秒，但用户提交的评价本身立即可见（`review` 已写入 DB），体验影响极小。如果要让当前用户"感觉"评分已经更新，可以在前端做乐观更新：提交评价成功后，前端本地计算新的平均分并临时显示，不等后端异步更新完成。等下次真正刷新商家主页时，从 DB/缓存拿到最新值覆盖。这种前端乐观更新配合后端最终一致，是常见的用户体验优化手段。

六、实时消息推送与通知提醒（含社区 Feed 流 + RabbitMQ）

Q19：聊天记录翻页为什么用游标分页（`WHERE id < lastId`），不用 `LIMIT offset`？

代码位置： `MessageServiceImpl.history()` ， `.lt(lastId != null, Message::getId, lastId).last("LIMIT " + (size+1))`

offset 分页（ `LIMIT offset, size` ）的问题：MySQL 在执行时会从头扫描 `offset` 行然后丢弃，翻到第 1000 页时实际扫了 10000 行，随着翻页数增大性能线性退化。

游标分页（ `WHERE id < lastId LIMIT size` ）： `id` 上有索引，直接定位到 `lastId` 的位置，每次查询都是 $O(\log n)$ 的索引查找，无论翻到第几页性能都一样。

聊天记录场景额外适合游标分页：用户从最新消息向上翻，是连续的"上一批"操作，`id` 单调递减，天然契合 `id < lastId` 的游标模式。

↳ 追问：游标分页有没有缺点？什么场景下反而应该用 offset 分页？

游标分页的缺点：无法跳转到指定页（"第 100 页"），不支持随机翻页，只能向前或向后连续翻。如果 `id` 不连续（有删除记录），`id < lastId` 的结果页大小可能不稳定。offset 分页适合以下场景：管理后台列表（需要"跳到第 N 页"）、报表分页（页码对用户有意义）、数据量有限且不会超过几百页（offset 扫描代价可接受）。聊天记录用户只会往上翻，不需要跳页，游标分页完全契合。

Q20：游标分页里怎么判断还有没有更多数据？为什么不做 COUNT 查询？

接 Q19，追问 `hasMore` 的判断方式。代码位置： `MessageServiceImpl.history()` 。

多查 1 条来判断：

```
.last("LIMIT " + (size + 1))           // 比需要的多查 1 条

boolean hasMore = messages.size() > size;
if (hasMore) messages = messages.subList(0, size); // 截掉多查的那条，只返回 size 条
```

如果实际返回了 `size + 1` 条，说明后面还有数据；只返回 `<= size` 条，说明已到末尾。

避免了额外的 `SELECT COUNT(*)` 查询（COUNT 在大表上也要全扫或走索引统计，不快）。代价是每次多传输 1 条数据，可以忽略不计。

↳ 追问：这里多查了 1 条数据，前端收到 `size` 条，但其实查了 `size+1` 条，这 1 条多余的数据会对性能有影响吗？

几乎没有影响。多查 1 条是在已经走了索引定位到 `lastId` 位置之后，额外读取 1 条数据，代价是 1 次额外的索引查找 + 1 次回表，耗时在微秒级。相比之下，`SELECT COUNT(*)` 需要扫描满足条件的所有行来计数，对于百万级聊天记录表是毫秒级的额外 IO。两种方案代价相差 2~3 个数量级，多查 1 条是更优的选择。

Q21：发消息时，DB 写入和 WebSocket 推送哪个先执行？顺序反过来会有什么问题？

代码位置： `MessageServiceImpl.send()` ， `save(message)` 在 `convertAndSendToUser(...)` 之前。

先写 DB，后推送：

```
save(message); // 先持久化
messagingTemplate.convertAndSendToUser(..., vo); // 后推送
```

如果先推送再写 DB：推送成功但 DB 写入失败，用户看到了消息，但刷新后消息不见了——消息"幻影"，体验非常差。

先写 DB 保证消息一定持久化。推送失败（用户离线）无所谓，用户上线后拉历史记录能看到这条消息，DB 是兜底。

↳ 追问：如果 DB 写入成功但 WebSocket 推送失败（比如用户刚好断网），这条消息会丢失吗？用户怎么看到它？

不会丢失。消息已存在 DB，WebSocket 推送仅是"实时通知"，是锦上添花。用户重新上线后，前端会触发一次历史消息拉取（`GET /api/message/history`），从 DB 拉取 `lastId` 之后的消息，这条"推送失败"的消息会出现在历史记录里。两者职责分离：DB 负责持久化（不丢失），WebSocket 负责实时性（可以失败）。这也是为什么项目选择"先写 DB 再推送"而不是"只推送"。

Q22：WebSocket 连接认证为什么不能复用 SecurityConfig 里注册的 JwtAuthenticationFilter？

代码位置： `StompAuthChannelInterceptor.preSend()` ，类注释里有完整解释。

`JwtAuthenticationFilter` 继承自 `OncePerRequestFilter`，是 `Servlet Filter`，只拦截 HTTP 请求——WebSocket 握手时走 HTTP Upgrade，之后所有 STOMP 帧通过 WebSocket 协议传输，不经过 `Servlet Filter` 链，也不经过 Spring MVC 的 `HandlerInterceptor` 链。

STOMP 的 `CONNECT` 帧是连接升级后客户端发的第一帧，在这里验证 token 是最自然的位置。

`StompAuthChannelInterceptor` 拦截 `CONNECT` 命令，从 `native header` 取 token，调 `JwtUtil.parse()` 验签解析得到 `userId`（纯计算，不依赖 `ThreadLocal`，线程安全），然后 `accessor.setUser(() -> userId)` 注入 `Principal`。

后续 `convertAndSendToUser(userId, ...)` 依赖这个 `Principal` 来路由到正确的会话，所以这里必须把 `userId` 注入为 `Principal`。

↳ 追问： `accessor.setUser(() -> userId)` 这里存的 `userId` 是 Long 还是 String？ `convertAndSendToUser` 是怎么用它路由到正确会话的？

`setUser` 接受 `java.security.Principal`（函数式接口），`getName()` 返回 `String`，所以存的是 `userId.toString()`。`convertAndSendToUser(userId, destination, payload)` 内部会查找所有 WebSocket 会话，找到 `Principal.getName().equals(userId)` 的会话，将消息投递到 `/user/{userId}/queue/notify`（Spring 自动拼

接)。如果同一用户开了多个 Tab，多个会话的 `Principal.getName()` 都等于该 `userId`，消息会被投递到所有会话——即多端同步收到通知。

Q30：帖子点赞为什么用 Redis 记录而不直接写 MySQL？ `setIfAbsent` 怎么实现 like/unlike 切换？

代码位置：`PostServiceImpl.toggleLike()`

如果用 MySQL 记录每条点赞（insert/delete），高并发下频繁写压力大，且查“是否点赞”需要 `SELECT WHERE post_id=? AND user_id=?`，有索引也要走 B+ 树。

Redis 方案实现真正的切换（like/unlike）：

```
String key = "post:likes:" + postId + ":" + userId;
Boolean firstLike = stringRedisTemplate.opsForValue()
    .setIfAbsent(key, "1", 24 * 365, TimeUnit.HOURS);
if (Boolean.TRUE.equals(firstLike)) {
    // key 不存在 → 首次点赞 → 写 key + 计数 +1
    lambdaUpdate().eq(Post::getId, postId)
        .setSql("like_count = like_count + 1").update();
} else {
    // key 已存在 → 已点赞，此次是取消 → 删 key + 计数 -1
    stringRedisTemplate.delete(key);
    lambdaUpdate().eq(Post::getId, postId)
        .setSql("like_count = GREATEST(like_count - 1, 0)").update();
}
```

`setIfAbsent` 是原子操作，返回 true/false 同时完成状态判断和写入，不存在并发下重复点赞的问题。
`GREATEST(like_count - 1, 0)` 防止计数出现负值。

缺点：Redis key 数量随帖子数和用户数增长为 $O(n \times m)$ ，长期来看需要考虑内存压力，可改用定期清理冷数据的策略。

↳ **追问：**`setIfAbsent` + MySQL 计数 +1 这两步不是原子的，如果 `setIfAbsent` 成功但 MySQL 更新失败，会怎样？

会出现状态不一致：Redis 说“已点赞”（key 存在），但 MySQL 的 `like_count` 没有增加。用户看到的点赞数不变，但再次点击会触发“取消点赞”逻辑。这是已知的一致性缺口。完善的方案有两条：① 把点赞事件写入 MQ，消费者串行处理“写 Redis + 更新 MySQL”，保证顺序和可重试；② 不在 MySQL 列存 `like_count` 冗余，而是查询时动态从 Redis 统计（SCAN key pattern），但这在 key 量大时性能差。项目当前规模下这个概率极低，可以接受，但生产级应当用 MQ 方案。

Q31：Feed 流有“关注的商家”和“全部”两个入口，用的是推模式还是拉模式？为什么？

代码位置： `PostServiceImpl.followedFeed()` 和 `PostServiceImpl.listAll()`

推模式 (Fan-out)：用户发帖时主动写入所有关注者的收件箱，查 Feed 时直接读。查询快，但写放大严重——大V发一条帖子要写入百万个收件箱。

拉模式 (Fan-in)：查 Feed 时实时拉取关注列表所有人的帖子合并排序。写只写一份，存储成本低，但读时有计算。

项目用的是**拉模式**：

```
// followedFeed: 先查关注的商家 ID 列表, 再 WHERE merchant_id IN (...)
List<Long> merchantIds = followMapper.selectList(...).stream()
    .map(Follow::getMerchantId).collect(Collectors.toList());
```

选拉模式的原因：Cosplay 社区里商家（内容生产者）数量远小于用户（消费者），每个用户关注的商家通常也就几十个，IN 子句的数据量可控；而推模式在这个场景下写放大严重——一个热门商家发帖要给几万粉丝各写一条，得不偿失。

↳ 追问：如果某个用户关注了 500 个商家，IN 子句有 500 个 ID，MySQL 会怎么处理这个查询？会走索引吗？

MySQL 会对 IN 子句里的 500 个值分别走 `merchant_id` 索引查找，找到所有匹配行后做合并和排序（通常需要 `filesort`，除非有 `(merchant_id, create_time)` 联合索引）。500 个 id 的 IN 查询在 MySQL 里是可接受的，但如果关注数增长到几千，IN 子句变长，索引查找次数线性增加，需要考虑优化：① 建 `(merchant_id, create_time DESC)` 联合索引，避免 `filesort`；② 超过一定关注数阈值的用户，异步预生成 Feed 列表存 Redis，查询时直接读（混合推拉模式）。

Q48：帖子图片上传是怎么实现的？为什么不经过后端服务器？

代码位置： `PostController.getUploadToken()` ，七牛云 Java SDK

流程（前端直传）：

- ① 前端：GET `/api/post/upload-token`
- ② 后端：调七牛云 SDK 生成 `UploadToken`（含 `bucket`、有效期 1 小时、文件名策略）
- ③ 前端：把图片 + token 直接 POST 到七牛云 CDN 上传节点（不经过后端）
- ④ 七牛云：返回文件 key（如 `UUID.jpg`）
- ⑤ 前端：把 key + 帖子内容一起提交到 `POST /api/post`
- ⑥ 后端：存 key，查询时拼接 CDN 域名（`application.yaml` 中的 `domain` 字段）返回完整 URL

为什么不让图片经过后端：

1. **带宽：**图片通常几 MB，后端中转会占用大量出入带宽，七牛云 CDN 节点更接近用户，速度更快
2. **连接数：**上传图片耗时较长（秒级），经过后端会占住 Tomcat 连接，高并发下连接池很快打满
3. **职责清晰：**后端只存 key，无状态，便于水平扩展；七牛云负责 CDN 加速和存储

安全性：token 由后端签发，有效期短（1 小时），且可以在 token 中限制文件类型和大小，防止恶意上传。

↳ 追问：前端拿到 upload token 后可以上传任意文件到七牛云，怎么防止滥用（比如上传可执行文件、超大文件）？

Token 里可以内置限制：① 文件类型限制——设置 `mimeLimit` 参数，只允许 `image/jpeg,image/png`，七牛云在接收文件时校验 MIME 类型，不符合则拒绝；② 文件大小限制——设置 `fsizeLimit` 参数，超过限制直接拒绝上传；③ Key 前缀策略——Token 里固定 key 前缀（如 `user/{userId}/`），用户只能上传到自己的目录下，无法覆盖他人文件；④ 短 TTL——Token 有效期 1 小时，过期后无法上传。后端存 key 时还可以校验 key 格式是否符合预期，防止前端伪造其他 bucket 的 key 注入。

Q32：项目里为什么用 MQ 做预约通知，而不是 booking 直接调 social 推 WebSocket？

代码位置：`MQSender.java` 注释、`BookingServiceImpl.confirm()`

直接调用（同步 Feign）的问题：

```
BookingServiceImpl.confirm()
    → Feign 调 mhp-social.pushNotify() ← 如果 social 宕机，这里抛异常
    → 事务回滚，预约确认失败
```

商家确认预约（核心业务）被通知推送（辅助功能）拖垮，两个不相关的功能强耦合在一条调用链上。

MQ 解决了三个问题：

1. **解耦**：booking 只管发消息，不关心 social 是否在线。social 宕机时消息积压在 MQ，恢复后继续消费，booking 的确认流程不受任何影响。
2. **异步**：booking 把消息塞进 MQ 即返回（< 1ms），不等通知发送完成。用户感知到的"确认"操作延迟极低。
3. **可靠**：MQ 持久化保证消息不丢。就算 social 宕机几小时，重启后所有积压的通知都能补发，用户最终一定能收到。

↳ 追问：你说 MQ 能"削峰"，这里的通知场景有削峰的需要吗？削峰、解耦、异步三者分别解决什么问题，不要混用？

通知场景主要用到**解耦**和**异步**，削峰需求不强。三者各自解决的问题：① **异步**：把慢操作移出请求链路，降低 RT。比如发通知不影响预约确认结果，可以异步做；② **解耦**：生产者和消费者独立部署、独立扩展，互不依赖。social 宕机不影响 booking，这是本场景的主要诉求；③ **削峰**：消费者按自己的速度消费，不被瞬时流量压垮。抢档期场景最典型——开放瞬间大量请求写入 MQ，消费者平滑处理，DB 不被直接冲击。三个作用是独立的，同一个 MQ 可以同时发挥多个作用，但引入 MQ 的核心理由要说清楚是哪个。

Q41：通知走 MQ 异步推送，和同步 Feign 调用 social 相比，RT 为什么更低？

接 Q32，追问 MQ 异步带来的延迟优势。

同步调用的 RT 是整条链路 IO 的总和：

```
confirm() → updateStatus(DB 写) → Feign 调 social.push() → social 推 WebSocket → 返回
      ↑_____ 用户等待所有步骤完成 _____↑
```

MQ 异步的 RT 只等最快那步：

```
confirm() → updateStatus(DB 写) → rabbitTemplate.send(MQ, <1ms) → 立即返回
                                   ↓
                               Consumer 异步处理: social 推 WebSocket
```

用户感知到的"确认预约"只需要等 DB 写入 (5~20ms) + 塞进 MQ (< 1ms)，不等通知发送完成。而同步方案还要加上跨服务 Feign 网络 RTT (几十 ms)。

核心思路：把"不影响结果正确性"的操作移出请求链路，响应速度由最慢的必要环节决定。

↳ 追问：RT 降低了，但通知的"实时性"也降低了——用户可能在确认后几十毫秒才收到通知，这是否可接受？如果要同时保证低 RT 和高实时性，有什么方案？

几十毫秒的延迟在 WebSocket 通知场景完全可接受——用户体感上"几乎是实时"。真正有问题的是 social 服务压力大时消息积压，延迟可能到秒级。如果要同时保证低 RT 和高实时性：① 优先保证消费者的处理速度，消费者不做耗时操作（只推 WebSocket，不做复杂业务），消费速度快则延迟低；② 消费者水平扩展，增加 Consumer 实例数消化积压；③ 用异步化的本地线程池代替 MQ（只在单服务内），把通知推送提交到线程池异步执行，RT 降低的同时没有网络传输延迟——但这牺牲了解耦，social 宕机时通知就真丢了。

Q42：confirm() 先改 DB 状态再发 MQ，如果 MQ 发送失败，通知丢了怎么办？为什么设计 Outbox？

代码位置：BookingServiceImpl.confirm()，updateStatus(bookingId, 2) 之后紧跟 mqSender.sendBookingConfirmed(...)。

真实问题：confirm() 没有 @Transactional，DB 状态已改为 2（已定档），但 rabbitTemplate.convertAndSend() 若抛异常，客人的 WebSocket 通知静默丢失，项目目前无任何重试或补偿机制。

Outbox 模式的核心思路：

本地事务内：

```
UPDATE booking SET status=2 WHERE id=?      ← 业务写
INSERT INTO outbox (payload, sent=false)    ← 消息登记
```

事务提交 →

独立 relay 服务 (Spring Scheduler 或专用进程)：

```
SELECT * FROM outbox WHERE sent=false
→ rabbitTemplate.send(...)
```

→ UPDATE outbox SET sent=true WHERE id=?

业务写和消息登记在同一个本地事务里，数据库 ACID 保证"要么都成功要么都失败"。relay 服务宕机重启后扫描 outbox 表补发，消息不丢。

	当前方案	Outbox 模式
DB 改状态成功，MQ 失败	通知丢失，无补偿	relay 扫描 outbox 重发，最终送达
消息持久化层	MQ（消息入 Queue 才算）	DB（outbox 记录即持久化）
实现复杂度	低	高（需 outbox 表 + relay 服务）

现状：项目是简化实现，通知丢失在生产环境是严重问题，引入 Outbox 是正确的待优化方向。

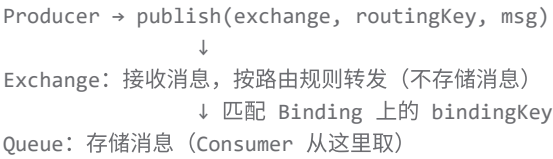
↳ 追问：Outbox relay 服务如果重启，从 outbox 表扫描重发时，已经发成功的消息可能被重发（at-least-once），Consumer 侧怎么保证不重复处理？

这正是 Outbox 模式要求 Consumer 侧必须幂等的原因。本项目 Consumer 已经用 Redis SET NX 做了幂等去重（见 Q35）：每条消息有唯一 msgId，setIfAbsent 保证同一 msgId 只处理一次。Outbox relay 重发时，消息的 msgId 不变（从 outbox 表的 payload 字段里带过来），Consumer 收到重复消息时 setIfAbsent 返回 false，直接 ACK 跳过。两者配合：Outbox 保证"至少一次投递"，Consumer 幂等保证"最多一次处理"，合起来实现"恰好一次"的业务效果。

Q33：RabbitMQ 的 Exchange、Queue、Binding 三者是什么关系？项目的 RabbitConfig 是怎么配置的？

代码位置：`mhp-booking/config/RabbitConfig.java`

三者关系：



项目拓扑：

```
schedule.exchange (Topic)
  bindingKey "notify.#" → notify.queue (x-dead-letter-exchange=schedule.dead.exchange)
                        | nack(requeue=false)
schedule.dead.exchange (Direct)
  bindingKey "notify.dead.queue" → notify.dead.queue
```

```
// Topic Exchange, durable=true (MQ 重启后不消失)
ExchangeBuilder.topicExchange("schedule.exchange").durable(true).build();

// 普通队列，绑定死信交换机
QueueBuilder.durable("notify.queue")
    .withArgument("x-dead-letter-exchange", "schedule.dead.exchange")
    .withArgument("x-dead-letter-routing-key", "notify.dead.queue")
    .build();

// Binding: 把 Queue 绑定到 Exchange 上，并指定 bindingKey
BindingBuilder.bind(notifyQueue).to(scheduleExchange).with("notify.#");
```

为什么选 Topic Exchange：生产者发多种类型通知（`notify.booking_confirmed`、`notify.rush_created` 等），用通配符 `notify.#` 一个绑定规则全部捕获，不需要为每种消息类型单独绑定。

为什么 booking 和 social 都要声明同一拓扑？

RabbitMQ 是"幂等声明"——重复声明参数相同的资源不会报错，不会创建重复。两个服务启动时各自声明一遍，谁先启动谁创建，后启动的那个声明直接通过，保证两个服务都不依赖对方先启动。

↳ 追问：Topic Exchange 的 * 和 # 有什么区别？`notify.#` 能匹配 routingKey 为 `notify` 的消息吗（没有点，只有 `notify`）？

* 匹配恰好一个词（词以 . 分隔），# 匹配零个或多个词。所以：`notify.#` 能匹配 `notify`（零个词）、`notify.booking`（一个词）、`notify.booking.confirmed`（两个词）。`notify.*` 则只匹配 `notify.booking` 这种恰好一个词的 routingKey，不匹配 `notify`（无词）或 `notify.booking.confirmed`（两个词）。项目用 `notify.#` 是为了未来可以扩展子类型（`notify.booking_confirmed`、`notify.rush_created`），一个绑定规则统一捕获所有通知类型，不需要为每种类型单独建 Binding。

Q34：手动 ACK 和自动 ACK 的区别是什么？项目为什么选手动 ACK？

代码位置：`mhp-social/application.yaml` → `acknowledge-mode: manual`，`NotifyConsumer.java`

自动 ACK：消息从 Queue 出队的瞬间自动确认。Consumer 取到消息后还没处理，进程崩溃 → 消息永久丢失，用户永远收不到通知。

手动 ACK：消息处理完后，Consumer 主动调 `channel.basicAck()` 才确认。处理前崩溃 → MQ 检测到连接断开 → 消息重新入队 → 下一个 Consumer 实例再次处理。

```
@RabbitListener(queues = NOTIFY_QUEUE)
public void onNotify(NotifyMessage msg, Message rawMsg, Channel channel) throws IOException {
    long tag = rawMsg.getMessageProperties().getDeliveryTag();

    try {
        doProcess(msg);
        channel.basicAck(tag, false); // 成功：确认，消息从 Queue 移除
    }
```

```

    } catch (Exception e) {
        channel.basicNack(tag, false, false);    // 失败：不确认，requeue=false → 死信队列
    }
}

```

basicNack 第三个参数 requeue :

- true : 消息重新入队 (适合网络抖动等临时错误)
- false : 消息不重入队, 路由到死信队列 (适合业务逻辑错误, 无限重试没意义)

项目选 requeue=false : WebSocket 推送失败通常是 Consumer 端的持久性问题, 重入队会无限循环, 不如进死信队列留存供运维分析。

↳ 追问: prefetch (QoS) 是什么? 为什么推荐在 Consumer 里设置 prefetch=1? 设太大有什么问题?

prefetch 控制 Consumer 最多同时持有多少条未 ACK 的消息 (即一次性预取数量)。设 prefetch=1 的效果: Consumer 每次只拿 1 条, 处理并 ACK 后再拿下一条, 消息在多个 Consumer 实例间均匀分发 (Round Robin)。设 prefetch=250 的问题: 某个 Consumer 实例一次性拿走 250 条, 即使它处理很慢 (比如处理的是耗时操作), 其他快的 Consumer 实例也只能等它 ACK 完这 250 条才能拿到新消息, 导致消息分布不均, 快实例空闲、慢实例积压。prefetch=1 牺牲一点网络往返 (每条消息都要 ACK 才取下一条), 换取消息在 Consumer 间的公平分发。

Q35: 消费者怎么保证同一条消息不被重复处理? (幂等性)

代码位置: NotifyConsumer.onNotify() 前半段

为什么会重复消费?

RabbitMQ 保证 At-Least-Once (至少一次) 投递: Consumer 处理后 crash 来不及 ACK → MQ 超时重投。网络抖动 MQ 没收到 ACK → MQ 重投。同一条消息可能被投递两次。

项目的幂等方案: Redis SET NX 去重

```

String idempotentKey = "msg:processed:" + msg.getMsgId();

Boolean isNew = stringRedisTemplate.opsForValue()
    .setIfAbsent(idempotentKey, "1", 24, TimeUnit.HOURS);

if (Boolean.FALSE.equals(isNew)) {
    // key 已存在 → 这条消息已处理过 → 直接 ACK 跳过
    channel.basicAck(tag, false);
    return;
}

```

msgId 由 MQSender 在发送时用 IdUtil.fastSimpleUUID() 生成, RabbitMQ 重投时 msgId 不变, 因此同一条消息无论被投递几次, setIfAbsent 只有第一次返回 true。

细节: 失败时要删 key

```

} catch (Exception e) {
    stringRedisTemplate.delete(idempotentKey); // 删除幂等 key
    channel.basicNack(tag, false, false);      // 进死信队列
}

```

处理失败时如果不删 key，下次重试（从死信队列手动重投）时 `setIfAbsent` 返回 `false`，消息被跳过，用户永远收不到通知。删 key 后允许下次重试时重新处理。

↳ 追问：幂等 key 的 TTL 设 24 小时，如果同一消息在 25 小时后被重新投递（比如死信手动重投），key 已过期，还会重复处理吗？有没有更强的保证？

会重复处理。24 小时 TTL 假设消息不会在 24 小时后被重投，对正常的 At-Least-Once 重试场景足够，但死信手动重投可能间隔很久。更强的保证：把 msgId 持久化到 MySQL 的 `processed_messages` 表（`msgId`, `process_time`），Consumer 处理前先查这张表。DB 的数据不会因 TTL 过期，可以永久去重。代价是每次消费都多一次 DB 读（可以加缓存）。另一种思路：让消费的业务操作本身幂等，不依赖外部去重——比如 WebSocket 推送相同消息，用户只是看到两条一样的通知，影响可接受。是否需要强幂等取决于重复消费的业务影响有多大。

Q36：如何保证消息从生产到消费全链路不丢失？三个环节分别怎么保证？

消息从 Producer 到 Consumer 经过三个环节，每个环节都可能丢消息：

```

Producer  —①— Exchange/Queue  —②— MQ宕机  —③— Consumer
  发送丢失      路由失败丢失      重启丢失      处理失败丢失

```

环节一：Producer → MQ（Publisher Confirm）

```
spring.rabbitmq.publisher-confirm-type: correlated
```

`correlated` 模式：每条消息独立绑定 `CorrelationData`，MQ 收到并处理后回调 `confirmCallback`（`ack=true`），无法路由到 Queue 时回调 `returnCallback`。

项目目前发消息后打日志，未实现重发逻辑（简化处理）。生产环境应在 `ack=false` 时写入重试表或告警。

环节二：MQ 自身持久化

三层缺一不可：

1. `Exchange durable(true)` — 重启后 Exchange 定义不丢失
2. `Queue durable(true)` — 重启后 Queue 定义不丢失，消息也不丢失
3. 消息 `deliveryMode=PERSISTENT` — Spring AMQP 默认即持久化，无需手动设置

环节三：Consumer 手动 ACK

见 Q34。处理完才 ACK，处理前 crash 消息重投，保证消息至少被成功处理一次。

↳ 追问：**publisher-confirm-type 设为 correlated 而不是 simple，两者有什么区别？**

simple 模式：整个 Channel 级别的确认，waitForConfirms() 会阻塞直到 Channel 上所有消息都被确认，无法区分哪条消息被确认哪条失败，不适合高并发场景。correlated 模式：每条消息绑定独立的 CorrelationData（携带消息 ID 等元信息），Broker 确认/拒绝时回调对应的 confirmCallback，精确知道哪条消息成功哪条失败，可以针对失败消息单独重发。项目用 correlated 是为了能精确重发失败消息，即使目前重发逻辑还没实现（只打日志），框架层面已经具备了精确重发的能力。

Q37：死信队列是什么？项目里是怎么设计的？

代码位置：RabbitConfig.java 注释、NotifyConsumer.java 的 basicNack

什么情况下消息变成死信：

1. Consumer basicNack / basicReject + requeue=false（处理失败且不重入队）
2. 消息在 Queue 里超过 TTL
3. Queue 达到最大长度（x-max-length），新消息被挤出

消息变成死信后，如果 Queue 配了 x-dead-letter-exchange，自动路由到死信交换机。

项目拓扑：

```
notify.queue 里的消息 basicNack(requeue=false)
    ↓ 自动路由
schedule.dead.exchange (Direct)
    ↓ routingKey 精确匹配 "notify.dead.queue"
notify.dead.queue（死信最终存放处）
```

死信队列的价值：

- 消息不丢失：处理失败的消息不会凭空消失，沉淀在死信队列
- 问题可追溯：运维可以查看死信队列里的消息内容，分析失败原因
- 人工重投：分析完问题后，可以把死信队列里的消息手动重新发到正常队列重新消费

↳ 追问：**死信队列里的消息如何实现自动延迟重试（比如失败后等 5 分钟再重试），而不是人工干预？**

利用 TTL + 死信的组合实现延迟队列：① 创建一个"等待队列"，不绑定 Consumer，设置 x-message-ttl=300000（5 分钟）和 x-dead-letter-exchange=正常交换机；② 消息处理失败后 nack 到这个等待队列（而不是直接进真正的死信）；③ 等待队列的消息 5 分钟后 TTL 过期，变成死信，自动路由回正常交换机 → 正常队列，重新被 Consumer 消费。这是 RabbitMQ 实现延迟重试的经典方案，无需插件。如果需要指数退避（第 1 次 1 分钟，第 2 次 5 分钟，第 3 次 30 分钟），可以建多个不同 TTL 的等待队列，Consumer 根据重试次数选择不同的等待队列路由。

Q38: RUSH_CREATED 和 RUSH_REMINDER 为什么在消费端做 fan-out，而不是生产端发多条消息？

代码位置：MQSender.sendRushCreated()（只发 1 条）、NotifyConsumer.onNotify()（查 follow 表遍历推送）

生产端 fan-out（备选方案）：MQSender 查出所有 follower，循环调用 N 次 rabbitTemplate.convertAndSend()，每位粉丝一条消息。

消费端 fan-out（项目采用）：MQSender 只发 1 条消息（带 merchantId），NotifyConsumer 收到后查 follow 表，拿到所有关注该商家的 userId，逐一推 WebSocket。

选消费端的原因：

1. **不跨服务查询：**follow 数据在 mhp-social，booking 服务查它需要跨服务调用。消费端在 social 服务里，直接查本服务的 follow 表，无额外跨服务依赖。
2. **发送可靠：**只需确认 1 条消息被 MQ 接收，Publisher Confirm 处理简单。生产端 fan-out 需要确认 N 条，哪条失败都需要重发。
3. **消息量小：**Queue 里只有 1 条，不会因为某个大V有 10 万粉丝就往 Queue 里塞 10 万条消息，避免队列瞬时膨胀。
4. **灵活扩展：**推送策略（是否跳过离线用户、是否合并推送）只需改 social 服务，booking 无感。

↳ **追问：**消费端 fan-out 时，如果一个商家有 10 万关注者，消费者要循环推 10 万次 WebSocket，这个消费者会成为瓶颈吗？怎么优化？

会。单个消费者串行推 10 万次 WebSocket，每次网络 RTT 算 1ms，光是推送就需要 100 秒，严重积压 MQ。优化方案分两层：① 当前服务内：把推送任务提交到线程池（ExecutorService），消费者只负责从 follow 表拿 userId 列表并提交任务，推送并行执行，整体耗时从“串行总和”降为“最慢一次”；② 横向扩展：把单条“商家发帖”消息转化为多条“给用户推送”消息，分发到多个 Consumer 实例并行消费——即在消费端做一次二级 fan-out，但这引入了消息数量膨胀问题，需要权衡。对于当前社区规模，线程池方案已足够。

七、按需搜索推荐（Elasticsearch + Function Score）

Function Score 是什么？是 ES 提供的功能吗？

function_score 是 Elasticsearch 的内置查询类型，不是项目自己实现的。它解决的问题是：ES 默认只按 BM25 相关度分（_score）排序，但很多业务场景需要把相关度与业务指标混合排序，function_score 就是专门做这个的——在原始查询的基础上，用自定义函数对每个文档的 _score 做二次加工。

结构：

```
{
  "function_score": {
    "query": { },           // 原始查询，产生 BM25 _score
    "functions": [ ],       // 自定义评分函数列表
    "score_mode": "sum",     // 多个 function 结果怎么合并 (sum/max/avg/min/multiply/first)
    "boost_mode": "multiply" // function 合并结果怎么与原始 _score 结合 (multiply/replace/sum/...)
  }
}
```

项目里的得分公式：

最终得分 = BM25_score × (avgScore × 1.2 + log1p(reviewCount) × 0.5)
 ↑ boost_mode=multiply ↑ score_mode=sum, 两个 function 加和

无关键词时所有文档 BM25 给相同的 1.0，等价于纯按业务指标排序；有关键词时相关度和业务质量共同影响排名。

ES 提供的 function 类型：

类型	用途
field_value_factor	用某个字段值参与算分（本项目用的）
script_score	用 Painless 脚本自定义算分逻辑
random_score	随机打散（用于随机推荐）
decay functions	距离/时间衰减（越近越高分，如地理位置搜索）

Q6：商家搜索为什么不用 MySQL LIKE，要引入 Elasticsearch？

代码位置： MerchantServiceImpl.search()

MySQL 的 LIKE '%keyword%' 有三个问题：

- 1. **全表扫描**： % 开头无法走索引，百万数据量下耗时秒级。
- 2. **无相关度**：LIKE 只返回是否匹配，无法按"keyword 和商家有多相关"排序，搜索体验差。
- 3. **不支持拼音**：搜索"zhuangniang"找不到"妆娘"，搜索"zn"也找不到。

ES 解决了这三个问题：倒排索引快速定位，IK 分词器+拼音子字段支持中文和拼音，BM25 算法计算相关度分数，天然支持按相关度排序。

↳ 追问：ES 的倒排索引具体是什么结构？和 MySQL 的 B+ 树索引相比，各自适合什么查询类型？

倒排索引的结构：词项字典（Term Dictionary，B 树或 FST）→ 词项对应的文档列表（Posting List，压缩存储文档 id、词频、位置）。查询"妆娘"时：在词项字典找到"妆娘"，O(log n) 直接拿到包含该词的所有文档 id，再做评分排序。B+ 树适合范围查询和等值查询（WHERE id BETWEEN 1 AND 100、WHERE city = '北京'），有序存储支持高效范围扫描。倒

排索引适合全文检索（`MATCH '妆娘'`），但不支持高效范围查询（倒排 Posting List 无序，范围查询需要合并多个列表）。两者互补：MySQL 做精确的结构化查询，ES 做全文搜索和相关度排序。

Q7：搜索排序是怎么设计的？为什么不直接 `sort(avgScore)` 而是用 `Function Score`？

代码位置：`MerchantServiceImpl.search()` 里的 `finalQuery` 构建

无关键词的情况（纯浏览）：应该按商家质量（评分、接单量）排序，相关度没有意义。

有关关键词的情况（有搜索意图）：应该相关度优先，但完全无视商家质量也不好——搜“妆娘”出来的第一个是差评累累的商家，用户体验差。

用 `sort` 的问题：两种场景需要不同的 `sort` 策略，切换逻辑复杂，且 `sort` 强制覆盖相关度，有关关键词时就无法利用 BM25 分数。

Function Score 统一解决：

最终得分 = BM25相关度分 × (`avgScore` × 1.2 + `log(1 + reviewCount)` × 0.5)

无关键词时 BM25 对所有文档给相同的 1.0 分，等价于纯按业务因子排序；有关关键词时相关度和业务质量共同影响排名，二者加权融合。一套算法覆盖两种场景。

↳ 追问：**BM25 是什么算法？它和 TF-IDF 相比改进了什么？**

TF-IDF：词频（TF）× 逆文档频率（IDF）。问题：① TF 无上限，词出现 1000 次比 100 次得分高 10 倍，但实际相关度不是线性的；② 长文档因为包含更多词而得分虚高。BM25 的改进：① 对 TF 做饱和处理（通过参数 `k1` 控制），词出现次数超过一定量后边际收益递减，避免“词频堆砌”刷高分；② 引入文档长度归一化（参数 `b`），长文档里的词不会因为文档长就被过度惩罚。实际效果 `BM25 > TF-IDF`，ES 5.0 起默认用 BM25 替代 TF-IDF。

Q8：Function Score 里为什么 `reviewCount` 用 `log1p` 处理，`avgScore` 直接线性？

接 Q7，追问公式里两个因子的处理方式不同的原因。

`reviewCount` 的问题是**量级差异**：有的商家 100 条评价，有的 10000 条，直接加权会让接单大户完全碾压新商家。

`log1p(x) = log(1+x)` 是边际递减函数——从 0 增长到 100 贡献大，从 1000 增长到 10000 贡献很小——合理反映规模效应同时给新商家机会。

`avgScore` 取值范围固定在 1~5，量级本身就均匀，直接线性加权即可。

另外，`avgScore` 的 `missing` 值设为 3.0（中等分），让没有评价的新商家也能参与排序，不至于永远排在末尾；`reviewCount` 的 `missing` 值设为 0，新商家不凭空获得接单量加分。

↳ 追问：`log1p` 的 "1p" 是什么意思？为什么不直接用 `log(reviewCount)`，而要 `log(1 + reviewCount)`？

"1p" 是 "1 plus" 的缩写，即 $\log(1 + x)$ 。直接用 `log(reviewCount)` 的问题：当 `reviewCount=0` 时， $\log(0)$ 是负无穷，会导致算分变成负无穷，使没有评价的新商家永远排在最末。 $\log1p(0) = \log(1) = 0$ ，没有评价的商家得 0 分，参与正常排序而不是被降到末尾。这也是为什么 ES 的 `field_value_factor` 里 `modifier` 专门提供了 `log1p` 选项——这是处理"从 0 开始计数的数值字段"的标准做法。

Q9：ES 里的数据是怎么和 MySQL 保持同步的？为什么不在 Service 层双写？

代码位置：`CanalSyncService.java`（`mhp-account` 启动后运行的后台线程）

双写方案（Service 层同时写 MySQL 和 ES）的问题：两次写操作不在一个事务内，MySQL 成功但 ES 失败时数据就不一致了，而且需要在每个写商家数据的地方都加 ES 写入，侵入性强、容易漏写。

Canal 方案：监听 MySQL binlog，从数据库层面感知所有变更，不依赖业务代码：

MySQL binlog → Canal Server → CanalSyncService（后台线程）→ ES

只需要保证 MySQL 写成功，Canal 保证 ES 最终一致。代价是有轻微延迟（毫秒~秒级），对商家搜索完全可接受。

Canal 还解决了一个难题：商家 ES 文档里的 `nickname` / `avatar` 来自 `user` 表，用户改昵称时 Canal 感知 `user` 表变更，找到对应商家并更新 ES doc，这是双写方案很难优雅处理的级联更新。

↳ 追问：Canal 是怎么感知 MySQL binlog 的？它伪装成了什么？为什么需要配置 `server-id`？

Canal 把自己伪装成 MySQL 从库（Replica），向 MySQL 主库发送 `COM_REGISTER_SLAVE` 命令注册，然后像真正的从库一样通过 `COM_BINLOG_DUMP` 请求 binlog 数据流，主库正常推送 binlog。MySQL 的主从复制协议要求每个从库有唯一的 `server-id`（32 位整数），用于标识节点，防止复制循环（ $A \rightarrow B \rightarrow A$ ）。Canal 配置的 `server-id` 必须在整个 MySQL 复制拓扑中唯一，如果设为 0 或与已有从库重复，MySQL 会拒绝注册。这也是为什么 CLAUDE.md 里特别提到"`server-id=0` 会被 MySQL 拒绝"——0 是 MySQL 的保留值。

Q10：CanalSyncService 里的 `ack/rollback` 是什么意思？处理失败了会怎样？

代码位置：`CanalSyncService.syncLoop()`，和 RabbitMQ 手动 ACK 是同一套思想。

```

while (!Thread.currentThread().isInterrupted()) {
    Message message = connector.getWithoutAck(100);    // 拉取但不推进游标
    long batchId = message.getId();

    if (batchId == -1 || message.getEntries().isEmpty()) {
        sleep(500);
        connector.ack(batchId);    // 空批也要 ack
        continue;
    }

    try {
        for (CanalEntry.Entry entry : message.getEntries()) {
            if (entry.getEntryType() != CanalEntry.EntryType.ROWDATA) continue;
            processEntry(entry);
        }
        connector.ack(batchId);    // 全部成功，游标往前移
    } catch (Exception e) {
        connector.rollback(batchId);    // 出错，下次重发整批
    }
}

```

ack：告诉 Canal "这批消费完了，游标往前移"，下次从新的 offset 开始。

rollback：告诉 Canal "这批没处理好，游标保持不动"，下次 `getWithoutAck` 重新从同一批开始，所有事件重发。

rollback 会把整批重发，因此 ES 操作必须幂等。 `merchantEsRepository.save()` 底层是 ES 的 index 操作，相同 id 会完整覆盖已有文档，无论执行多少次结果相同，天然满足幂等要求。

↳ 追问：rollback 会把整批 100 条重发，如果是第 50 条失败，前 49 条已经成功写入 ES，重发后这 49 条会被重复写入，会有数据问题吗？

不会有数据问题，但有性能开销。ES 的 index 操作幂等：相同 id 的文档再次 index 会完整覆盖，无论执行多少次，最终状态与只执行一次完全相同，不会产生重复文档。代价是这 49 条会被重复处理，增加 ES 的写负载。如果频繁失败（比如第 50 条总是失败），会造成 Canal 卡死在这个 batchId、前 49 条被反复重写的问题。此时应该在 `processEntry` 内部捕获个别条目的异常，记录失败的 entry 但不让整批 rollback，成功处理其他条目后再 ack。这是"跳过坏消息"策略，代价是失败的那条记录会丢失同步，需要人工补偿。

Q11：用户创建评价后，ES 里的 avgScore 和 reviewCount 是怎么更新的？

代码位置： `ReviewServiceImpl.create()` → `RpcMerchantServiceImpl.updateScore()` → Canal → `CanalSyncService.handleMerchantChange()`

`avgScore` 和 `reviewCount` 并不是 ES 独立维护的字段，它们源自 MySQL `merchant` 表的冗余列，经由 Canal CDC 同步到 ES：

- ① 用户提交评价, ReviewServiceImpl.create()
- ② 全量重算该商家的平均分和评价数
- ③ Dubbo RPC 跨服务写回 MySQL → UPDATE merchant SET avg_score=?, review_count=? WHERE id=?
→ 同时删除 Redis 缓存 (Cache Aside 写策略)
- ④ MySQL 产生 binlog
- ⑤ Canal Server 捕获 binlog, 推给 CanalSyncService
- ⑥ handleMerchantChange() 重查 DB 拿完整行
- ⑦ merchantEsRepository.save(doc), 更新 ES

为什么不直接用 binlog 的 afterColumns, 而要重查 DB:

ROW 格式 binlog 的 afterColumns 只记录本次 UPDATE 改动的列, 未改动的列不出现在里面。如果只用 afterColumns 构建 MerchantDoc, 其他未变的字段 (如 nickname、city) 就会是 null, 写进 ES 后会把已有数据覆盖掉。因此策略是: 只从 binlog 取 id, 再用 id 重查 MySQL 拿完整行, 构建完整的 MerchantDoc 后同步到 ES。

↳ 追问: Canal 是异步的, 评价提交后 ES 里的评分什么时候能更新? 这段时间用户搜索到的是旧评分, 会有问题吗? 如果要求评价后搜索结果立刻反映新评分, 怎么做?

Canal 延迟通常毫秒到秒级, 用户刚提交评价, 短暂看到旧评分是正常体验, 不影响业务。如果要立刻反映, 有两条路:

① 在评价创建成功后, 除了写 MySQL, 同时直接调 ES SDK 更新该商家的 avgScore 字段 (UpdateRequest, 只更新这一个字段, 不是全量覆盖)——这是双写, 引入了 MySQL 和 ES 不一致的风险, 但实时性最好; ② 接受延迟, 在前端做乐观更新: 提交评价成功后, 前端本地临时显示新的预估评分, 等下次真正刷新商家详情页时从后端拿到最新值。项目当前搜索排序对评分变化不敏感 (几秒延迟不影响搜索结果位置), Canal 异步方案已足够。

需要掌握的追问点

模块一: 商家主页 Redis 缓存

- ☐ 穿透/击穿/雪崩三者区别与解决方案
- ☐ Cache Aside 删除 vs 更新缓存的数据一致性分析
- ☐ DCL 两次检查各自的作用
- ☐ 空值哨兵 TTL 为什么要短 (2分钟)
- ☐ 随机 TTL 为什么能防雪崩
- ☐ 为什么不引入 Caffeine 二级缓存: 实例数少时 Redis 未成瓶颈, 引入后需解决跨实例缓存失效同步 (MQ 广播 + TTL 兜底 + 重启时清空本地缓存)

模块二: Spring Security + JWT 认证与鉴权

- ☐ Cookie vs Header token 安全性对比 (CSRF vs XSS)
- ☐ ThreadLocal 的生命周期与线程池污染问题 (Spring Security 的 SecurityContextHolderFilter 自动清理)
- ☐ JWT 三段结构 (Header.Payload.Signature)、HS256 验签原理、过期机制 (exp 字段)
- ☐ JWT 无状态的代价: token 过期前无法主动失效; 如需支持强制下线, 需引入 Redis 黑名单

- ☐ JWT 与 Sa-Token 对比：JWT 无状态、纯计算验证；Sa-Token 有状态、Redis 存储、可主动失效
- ☐ MD5 vs BCrypt 安全性对比（彩虹表攻击）

模块三：在线预约与档期维护

- ☐ Redisson tryLock 三个参数含义：waitTime/leaseTime/TimeUnit
- ☐ finally 里为什么要先判断 isHeldByCurrentThread() 再 unlock
- ☐ lock key 为什么同时包含 userId 和 scheduleId（锁粒度最小化）
- ☐ 状态机非法流转的防御：每次变更前校验当前状态
- ☐ status=1（待付款）为什么保留但不使用（预留支付接入的扩展点）
- ☐ XXL-Job vs @Scheduled 在多实例部署下的行为差异
- ☐ 问卷答案存 JSON 列 vs 单独建答案表的取舍

模块四：抢档期排队

- ☐ Redis Lua 脚本的原子性来源（单线程执行器）
- ☐ ZSET 的 ZADD/ZCARD/ZSCORE/ZRANK 时间复杂度（均为 $O(\log n)$ ）
- ☐ 返回值 -1/0/正整数的语义设计
- ☐ Redis 和 MySQL 的最终一致性保证
- ☐ Redis 入队后 MySQL 写失败的补偿方案（ZREM 撤回 vs Outbox 模式的适用场景）
- ☐ 缓存预热：rushOpenTime 已知 → XXL-Job 提前预热 + ApplicationRunner 启动兜底
- ☐ 热点识别两类思路：运行时探测（redis-cli --hotkeys）vs 可预期热点主动预热

模块五：微服务架构

- ☐ Dubbo vs Feign 选择依据（强依赖 vs 弱依赖）
- ☐ OpenFeign 的 batchGet 为什么要用（N+1 查询问题）
- ☐ 调用方向为 DAG 的好处（故障隔离、启动顺序）
- ☐ Nacos 服务注册的心跳机制（默认 5s, 15s 不续约视为不健康）
- ☐ @Transactional 只管本服务 DB，跨服务调用不在事务内（最终一致 vs 强一致代价）
- ☐ 全量重算 avgScore vs 增量计算的取舍（简单换正确）

模块六：实时消息推送与通知提醒

- ☐ 游标分页 vs offset 分页的性能差异（索引利用率）
- ☐ 多查一条判断 hasMore 的原理
- ☐ STOMP CONNECT 帧鉴权为什么不能用 HandlerInterceptor
- ☐ convertAndSendToUser 的路由依赖 Principal
- ☐ 先写 DB 再推 WS 的顺序原则
- ☐ 点赞 Redis key 的 $O(n \times m)$ 内存增长问题及应对

- ☐ 推模式 vs 拉模式的选型依据（生产者规模 vs 消费者规模）
- ☐ 七牛云直传：为什么不经过后端服务器（带宽 + 连接数 + 无状态）
- ☐ MQ 三大作用：异步、解耦、削峰——各自解决什么问题
- ☐ Exchange 四种类型（Direct/Topic/Fanout/Headers）及适用场景
- ☐ Topic Exchange 通配符语义： * 匹配一个词， # 匹配零或多个词
- ☐ 消息不丢失三环节：Publisher Confirm / MQ 持久化（exchange+queue+message） / 手动 ACK
- ☐ basicAck / basicNack / basicReject 参数含义， requeue=false 触发死信
- ☐ 死信队列触发条件（nack+requeue=false / TTL过期 / 队列满）及作用
- ☐ 消费者幂等设计：Redis SET NX 去重，失败时为什么要删 key
- ☐ At-Least-Once 投递语义 vs Exactly-Once 的实现代价
- ☐ 消费端 fan-out vs 生产端 fan-out 的权衡（跨服务查询 / 消息量 / 发送可靠性）
- ☐ Outbox 模式：本地事务同时写业务表 + outbox 表，relay 扫描发 MQ，解决 DB 成功后 MQ 失败导致通知丢失
- ☐ 同步调用 vs MQ 异步的 RT 差异：链路所有 IO 之和 vs 只等最快那步

模块七：按需搜索推荐

- ☐ IK 分词器工作原理（ik_max_word vs ik_smart）
- ☐ BM25 相关度分算法与 TF-IDF 的区别
- ☐ Function Score 的 scoreMode（sum/max/avg）与 boostMode（multiply/replace）
- ☐ Canal CDC 与双写的对比（一致性 vs 侵入性）
- ☐ binlog afterColumns 为什么不可靠，为什么要重查 DB
- ☐ MySQL binlog ROW 格式 vs STATEMENT 格式区别
- ☐ "推荐"边界说明：基于评分/相关度加权排序，不是个性化推荐算法
- ☐ RPS（每秒请求数） / RT（响应时间 P50/P99） / 错误率：压测三要素